MCT-454L Mobile Robotics

Lab Manual: ROS 2 Workspace Setup, Package Creation, and GitHub Introduction

Lab Title: Setting Up a ROS 2 Workspace, Creating a New Package, and Controlling Turtlebot in a Specific Pattern

Objective:

1. Set up a ROS 2 workspace.
2. Learn how to use GitHub for version control.
3. Learn how to create a new ROS 2 package.
4. Write and edit a basic node to move the turtle in a specific pattern.

Prep

Task 0.1 — Cleanup. Close any open windows and terminals. Remove directory ~/ros2_ws if it exists. This might be leftover from a previous session.

Task 0.2 — GitHub Account. Use the following commands to logout of any existing GitHub accounts, and login to the account of one of your group members.

Cheat Sheet

```
gh auth logout # Log out of any accounts from previous sections. gh auth
login # Login to the account your group is using. # (1) press enter to
select Github.com # (2) select press enter to select HTTPS # (3) type Y
then hit enter # (4) Take note of the supplied code, then press enter to
open # the web browser. Follow the instructions in the browser to #
login to your GH account. # (5) Once you've logged in, return to the
terminal and press enter # to finalize the authentication process.
```

Part 1: Setting Up a ROS 2 Workspace

Step 1: Open a Terminal and Create a New ROS 2 Workspace

Open a terminal (Ctrl + Alt + T) and run the following commands:

```
# Create a new ROS 2 workspace directory and a 'src' folder inside it
mkdir -p ~/ros2_ws/src
# Navigate to the workspace directory
cd ~/ros2_ws
# Build the workspace
colcon build
```

Step 2: Source the Workspace

```
# Source the workspace so that ROS 2 recognizes it
source install/setup.bash
```

To automatically source the workspace every time a terminal is opened, add the following line to `~/.bashrc`:

```
# Add workspace sourcing to .bashrc for automatic loading
echo 'source ~/ros2_ws/install/setup.bash' >> ~/.bashrc
# Apply the changes
source ~/.bashrc
```

Part 2: Using GitHub for Version Control

Step 1: Open a Terminal and Install Git (If Not Installed)

```
# Install Git if it is not already installed
sudo apt install git
```

Step 2: Configure Git

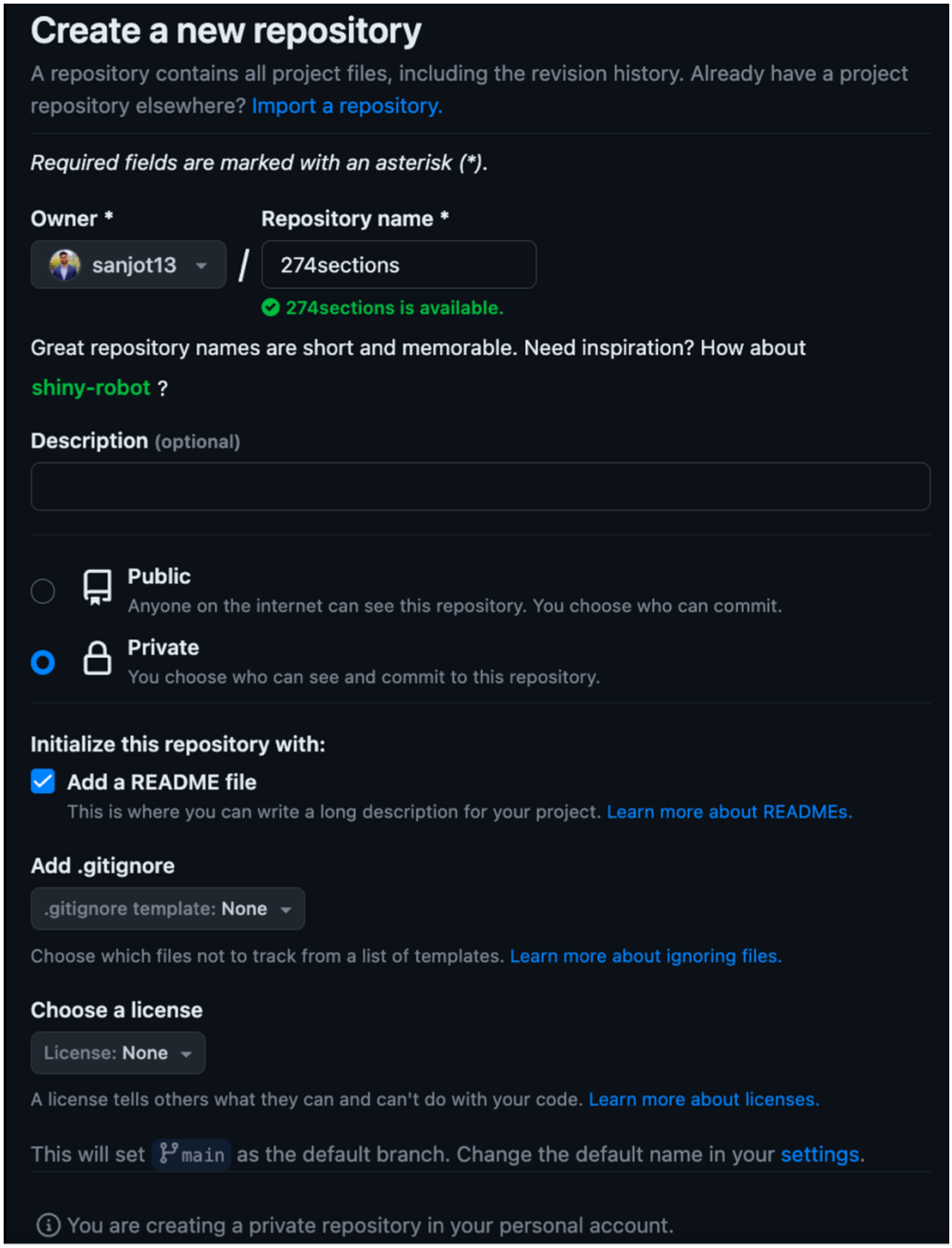
```
# Set your Git username
git config --global user.name "Your Name"
# Set your Git email
git config --global user.email "your_email@example.com"
```

Step 3: Initialize a Git Repository

```
# Navigate to the 'src' folder of your ROS 2 workspace
cd ~/ros2_ws/src
# Initialize Git in this directory
git init
```

Step 4: Create a GitHub Repository

1. Go to GitHub
2. Create a new repository.
3. Copy the repository URL.



Step 5: Connect Local Repository to GitHub and Push Changes

```
# Add the GitHub repository as a remote repository
git remote add origin <your-repository-url>
# Set the main branch
git branch -M main
```

Once you have completed your work, add, commit, and push all changes to GitHub:

```
# Add all files to Git
git add .
# Commit the changes
git commit -m "Final commit for lab session"
# Push the changes to GitHub
```

```
git push -u origin main
```

Part 3: Creating a New ROS 2 Package

Step 1: Open a Terminal and Navigate to ROS 2 Workspace

```
# Navigate to the 'src' folder of your ROS 2 workspace
cd ~/ros2_ws/src
```

Step 2: Create a New Package

```
# Create a new ROS 2 package named 'my_turtle_package' using Python
ros2 pkg create --build-type ament_python --dependencies rclpy turtlesim --
node-name my_node my_turtle_package
```

Step 3: Navigate to the Package Directory

```
# Move into the package directory
cd my_turtle_package
```

Step 4: Modify `setup.py` and `package.xml`

Ensure that `setup.py` correctly includes your executable script.

Modify `package.xml` to include dependencies such as `rclpy` and `turtlesim`

You can also modify these files later to add more nodes and dependencies.

Part 4: Creating a Node to Move the Turtle in a Pattern

Step 1: Navigate to the Scripts Directory

```
# Navigate to the package directory
cd my_turtle_package (Note that this directory is inside my_turtle package
directory with the same name.

All the .py files will reside here.
```

Step 2: Create a Python Script

Method 1:

```
# Create a new Python script named 'move_turtle.py'
touch move_turtle.py
# Make the script executable
chmod +x move_turtle.py
```

Method 2 (recommended):

You can also install visual studio for writing you're your scripts with the following command.

```
# Navigate to the src directory
# Install visual studio code
```

```
Sudo snap install code --classic

# Open VS code
Code .
```

Step 3: Edit `move_turtle.py` to Control Turtle

Open `move_turtle.py` and add the following code:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg

import Twist from std_msgs.msg

import String import time

class VelocityPublisher(Node):

      def __init__(self):
          super().__init__('velocity_publisher')
          self.publisher_ = self.create_publisher(Twist, 'turtle1/cmd_vel',
      10)

          timer_period = 0.5 # seconds

          self.timer = self.create_timer(timer_period, self.timer_callback)
      def timer_callback(self):

            msg = Twist()
            #for _ in range(4): # Move in a square msg.linear.x = 2.0 #
            Move forward msg.angular.z = 0.0 self.publisher_.publish(msg)
            time.sleep(2)
            msg.linear.x = 0.0
            msg.angular.z = 1.57 # Turn 90 degrees
            self.publisher_.publish(msg) time.sleep(1)
            #msg.linear.x = 0.0
            #msg.angular.z = 0.0 #self.publisher_.publish(msg)

      def main(args=None):

      rclpy.init(args=args)

      velocity_publisher = VelocityPublisher()
      rclpy.spin(velocity_publisher)

      # Destroy the node explicitly
      # (optional - otherwise it will be done automatically # when the
      garbage collector destroys the node object)
      velocity_publisher.destroy_node()
      rclpy.shutdown()

if __name__ == '__main__': main()
```

Step 3: Control Turtle with this Script

```
# Navigate to the workspace directory
cd ~/ros2_ws

# Build the workspace
colcon build

# Source the workspace again so that ROS 2 recognizes the new pckage

source install/setup.bash

# Run Turtlesim_node and my_node in two separate terminals

ros2 run name_of_the_package name_of_the_node
```

Task:

Modify the script to:

1. Move the turtle in a circular and triangular pattern.
2. Spawn three turtles and move them in different patterns.
3. Move the turtle to a specific location. *(Hint: Use the appropriate ROS 2 topics to control the turtle's position.)*